



LATER GATOR

SOFTWARE ARCHITECTURE MODELS

PREPARED FOR

CEN3031 – Introduction to Software Engineering

FALL 2025 | University of Florida

Professor Lisha Zhou

PREPARED BY

TEAM LATER GATOR:

Danielle Foege

Madison Holt

Carrie Ruble

Samantha Wong

Table of Contents

- SOFTWARE ARCHITECTURE DESIGN 1
 - Architectural Pattern Selection 1
 - Decomposition and Component Overview 1
 - Design Structures and Layers..... 2
 - Presentation Layer:..... 2
 - Application Layer:..... 2
 - Business and Logic Layer: 2
 - Data Layer: 2
 - External Services/Cross-Cutting Layer: 2
- TECHNOLOGIES AND TOOLS 3
 - Frameworks and Libraries 3
 - APIs and External Integrations 3
 - Development Tools 3
- SYSTEM CONTEXT MODEL 5
- CONCLUSION 6
- APPENDIX..... 7
 - A. Architecture Diagrams 7
 - 1. Swim Lane Diagram..... 7
 - 2. Wireframe Navigation Map..... 8
 - 3. Layered Architecture Diagram 9
 - 4. Component Decomposition Diagram..... 10
 - B. System Context Model 11

SOFTWARE ARCHITECTURE DESIGN

In exploring software architecture options for the Later Gator prototype, the team compared several design approaches, including microservices, client-server, and event-driven architectures. Because our app runs entirely on a single device and does not rely on distributed services, those models were unnecessarily complex for our goals. The chosen pattern, offers a more practical and maintainable fit, allowing us to keep components modular while simplifying development and deployment for a mobile environment.

Architectural Pattern Selection

The team selected a layered architecture pattern implemented within a monolithic style. This combination provides clear separation of concerns while keeping deployment simple for a single-device mobile app. Each layer has distinct responsibilities that support scalability and ease of modification as new features are introduced.

Decomposition and Component Overview

The Later Gator team began the decomposition process by exploring how users would interact with the app through a series of swim lane diagrams. This approach allowed us to visualize the responsibilities of each layer (presentation, application, business logic, and data access) while experimenting with user flows before any code structure was finalized.

As our understanding deepened, we transitioned from abstract flows to wireframe-based decomposition, focusing on how interface elements such as graphs, buttons, text boxes, and pop-ups connect to system logic and stored data.

These iterations (Appendix A1, A2) illustrate how the team learned to break down a mobile system into layered, maintainable components, bridging the conceptual model to an implementable design. The team identified system responsibilities, visualized workflow, and defined connections between interface elements, business logic, and stored data.

The resulting structure, a layered architectural diagram (Appendix A3), shows how these components were refined into maintainable layers. The resulting component decomposition diagram (Appendix A4) also builds on those earlier iterations by showing the finalized structure of the Later Gator system. It highlights the relationships among components, their placement within architectural layers, and how data flows between them, capturing the complete design logic that guides our implementation phase.

Design Structures and Layers

From this decomposition, the team established a structured set of layers that organize Later Gator's functionality. Each layer manages a specific concern: the presentation of user interfaces, coordination of application behavior, enforcement of business rules, management of data access, and connection to external services. This structure supports modularity, efficient troubleshooting, and easier adaptation as new features are added. The following subsections describe the responsibilities of each layer in detail.

Presentation Layer:

This layer deals with the user's interactions and the interface. It includes displaying notifications, pop-ups, onboarding, reports, and reflections. React Native provides for simple development as it includes easy-to-use modules.

Application Layer:

This layer deals with app activity and usage. It includes the Pomodoro Study Mode, snooze controls, app blocking manager, and settings. It also helps orchestrate data flow, decision-making, and coordination of requests between the Presentation and Business layers.

Business and Logic Layer:

This layer deals with the underlying decisions and functionality of the app. It includes setting limits to restrict phone usage, deciding what happens when a user takes action, managing notifications, and checking thresholds. This compartmentalization improves maintainability of app features.

Data Layer:

This layer deals with data access. It includes reading local user data and reading/writing to the Later Gator database tables. The database holds information related to the user, time spent on apps, snooze/study logs, reflections, app limits, and other settings.

External Services/Cross-Cutting Layer:

This layer deals with the security of the app as well as the user's privacy and performance. It includes a secure login and permissions.

TECHNOLOGIES AND TOOLS

In selecting technologies for the Later Gator prototype, the team prioritized tools that would simplify development, support modular architecture, and integrate smoothly within a mobile environment. Each choice reflects a balance between functionality, maintainability, and accessibility for a small development team.

Frameworks and Libraries

Alternative frameworks like Flutter or Swift were considered, but React Native was chosen for its cross-platform flexibility, extensive documentation, and simple component structure. It supports fast prototyping and aligns with the layered architecture by separating user interface concerns from underlying logic.

Room and SQLite were selected for local data persistence, providing reliable access to stored app and user data without external dependencies.

Technologies Used:

- **React Native:** Handles presentation and navigation through reusable UI components, simplifying screen transitions and front-end logic.
- **Room/SQLite:** Supports the data layer through reliable local storage, efficient queries, and lightweight setup for offline use.
- **Kotlin + Android Studio:** Provides direct access to Android APIs (such as *UsageStatsManager*) for evaluating usage statistics and generating reports.
- **Firebase:** Used in the cross-cutting layer for secure authentication and user sign-in with Google integration.

APIs and External Integrations

APIs were intentionally kept minimal to maintain the simplicity of a monolithic prototype. Firebase is the primary integration, providing authentication services and user management. Expo and React Native modules handle device notifications, background timers, and permission requests, connecting the business logic layer to native mobile features.

Development Tools

To ensure consistency across the development process, the team uses collaborative tools that support version control, testing, and deployment automation.

Development & Quality Tools:

- **GitHub:** Centralized repository for version control and peer review.

- **CircleCI:** Continuous integration pipeline used for automated testing and build verification.
- **Jira:** Task management and sprint planning, ensuring progress aligns with project milestones.

Together, these tools promote transparency, maintain code quality, and streamline coordination between team members working on different layers of the architecture.

The combined technology stack aligns with Later Gator's goal of delivering a lightweight, fully local mobile app prototype. Each tool supports a specific architectural layer, ensuring that design decisions translate directly into implementation. This cohesive environment enables rapid testing, efficient debugging, and a consistent user experience across all features.

SYSTEM CONTEXT MODEL

The system context model defines how the Later Gator application interacts with its surrounding environment. It provides a high-level view of all external entities that exchange data with the system, highlighting where boundaries exist between internal processes and outside services.

Later Gator operates as a standalone mobile app, with all core features contained within a monolithic, layered architecture. The app runs locally on the user's device, connecting only to a few external services for authentication and data handling. The diagram in Appendix B illustrates these relationships.

- **End User:** Interacts directly with the application through the React Native interface. The user logs in, sets time limits, manages snoozes, and views usage reports.
- **Firebase:** Manages authentication and secure sign-in. It enables user identification and protects data access across sessions.
- **Room Database (SQLite):** Handles local data storage. It reads and writes user data such as snooze attempts, session logs, study reflections, and app-limit settings.
- **React Native Framework:** Serves as the bridge between the user interface and the device's native operating system features, including notifications, timers, and background tasks.

These interactions form a closed ecosystem in which Later Gator captures and stores behavioral data securely on-device while communicating with Firebase only for login validation. This design reinforces user privacy, minimizes dependency on external networks, and supports the layered pattern by isolating data, logic, and interface responsibilities within the system boundary.

CONCLUSION

The Later Gator architecture reflects the team's focus on creating a structured yet approachable foundation for a locally deployed mobile app. By choosing a layered pattern within a monolithic style, we balanced separation of concerns with implementation simplicity, ensuring that each part of the system, user interface, logic, and data, can evolve without disrupting the whole.

The design and tool choices align with the goal of delivering a reliable prototype that leverages local data storage, built-in device capabilities, and simple authentication. The app avoids unnecessary external dependencies, emphasizing privacy and smooth performance for end users. Wrapping all of these features in an engaging and easy-to-use interface makes Later Gator an appealing solution for individuals looking to reduce their screen time.

Throughout this process, our team learned to move from abstract system concepts to practical design decisions. Experimenting with decomposition methods, wireframes, and context models helped us visualize how user actions translate into data flow and system behavior. These iterative steps strengthened both our understanding of software architecture and our ability to communicate design reasoning collaboratively.

The resulting model positions Later Gator for scalable growth while maintaining clarity and cohesion in its early prototype stage. As development continues, this structure will support easier troubleshooting, modular feature expansion, and improved team collaboration across future sprints.

APPENDIX

A. Architecture Diagrams

1. Swim Lane Diagram

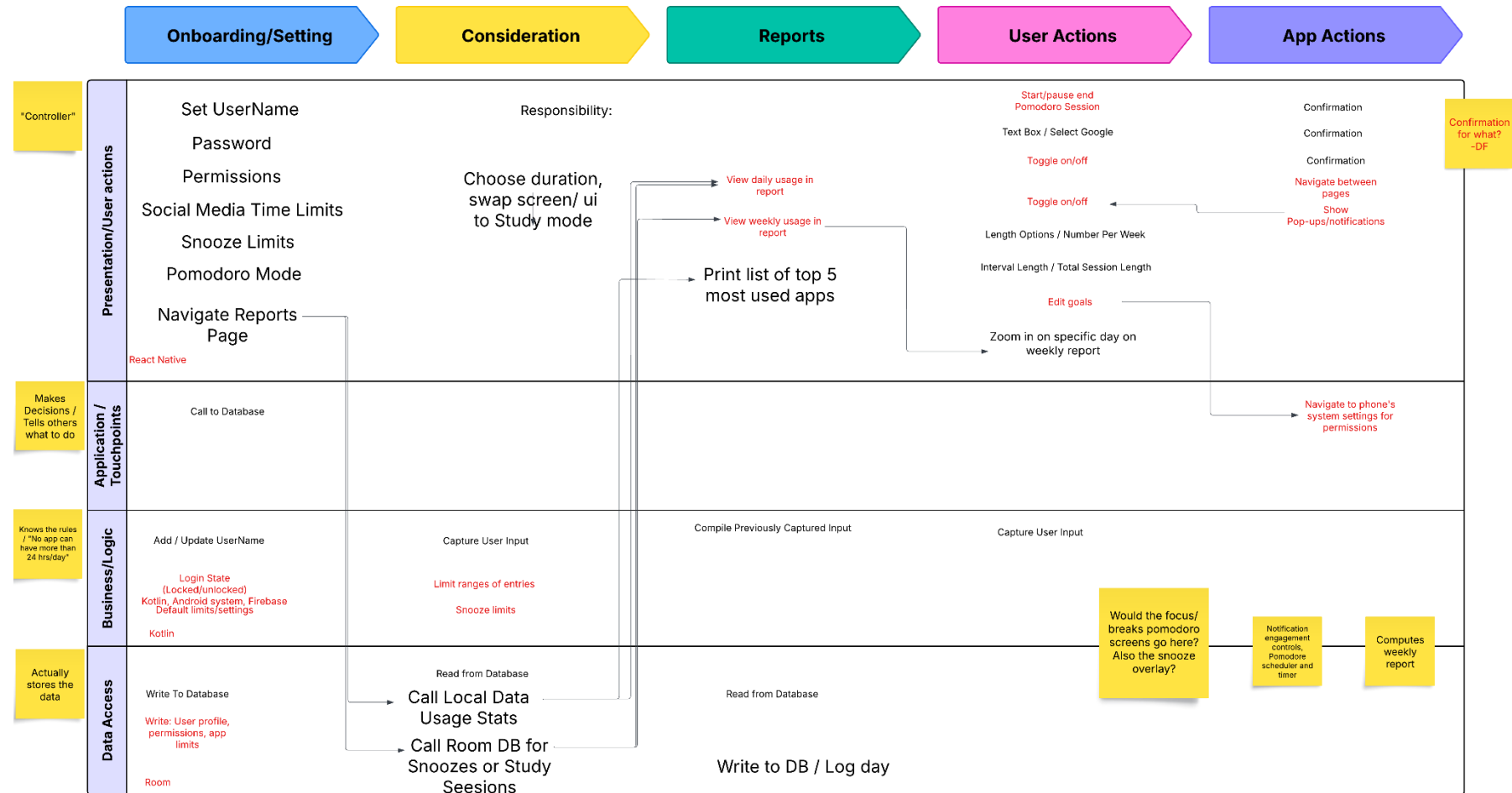


Figure A1: Early conceptual view showing how user actions pass through interface, logic, and data responsibilities. This initial mapping helped the team visualize workflow before finalizing the layered structure.

2. Wireframe Navigation Map

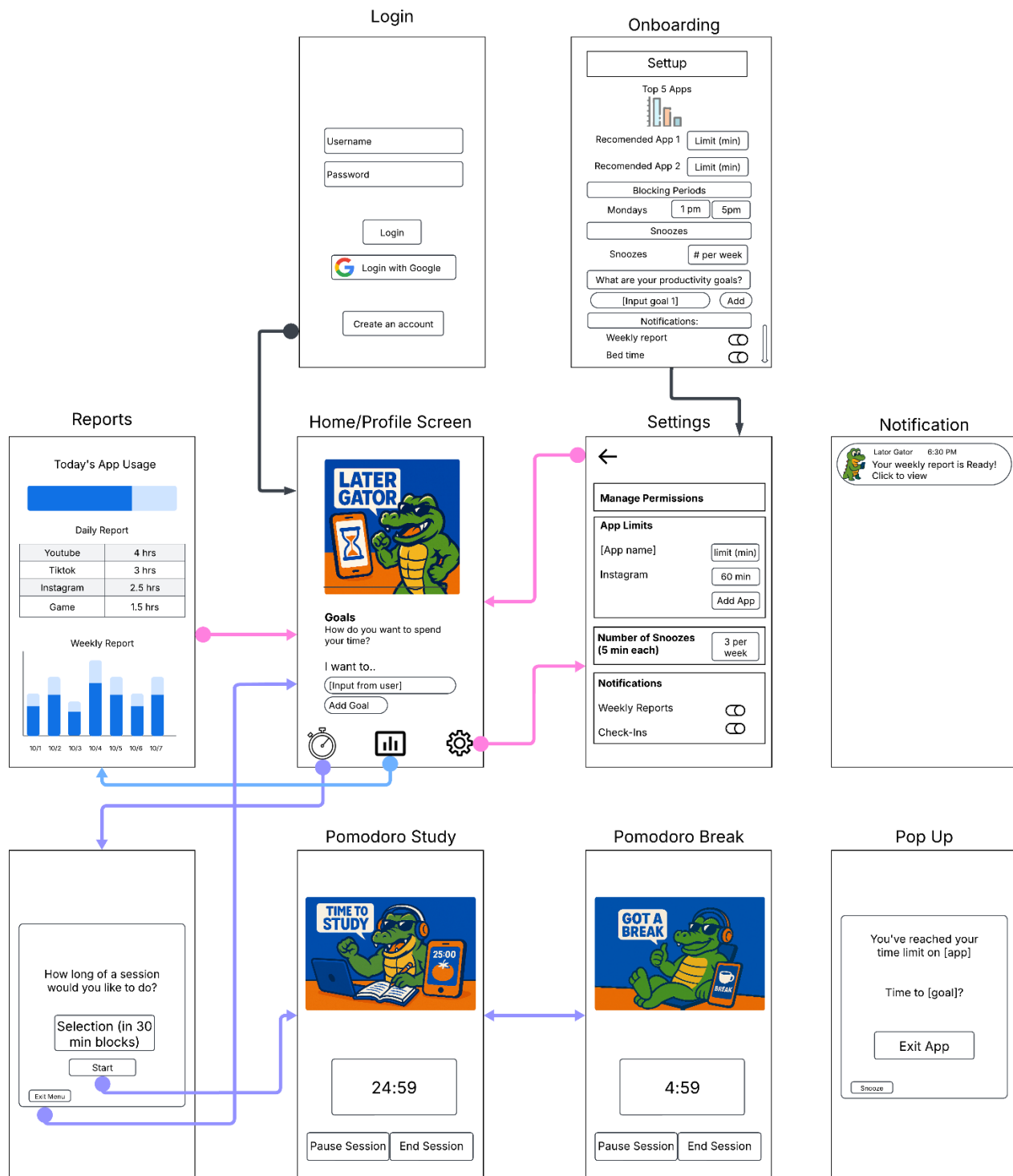


Figure A2: Wireframe model outlining the primary user interfaces and navigation flow. This diagram guided design decisions for component grouping and user interaction patterns.

3. Layered Architecture Diagram

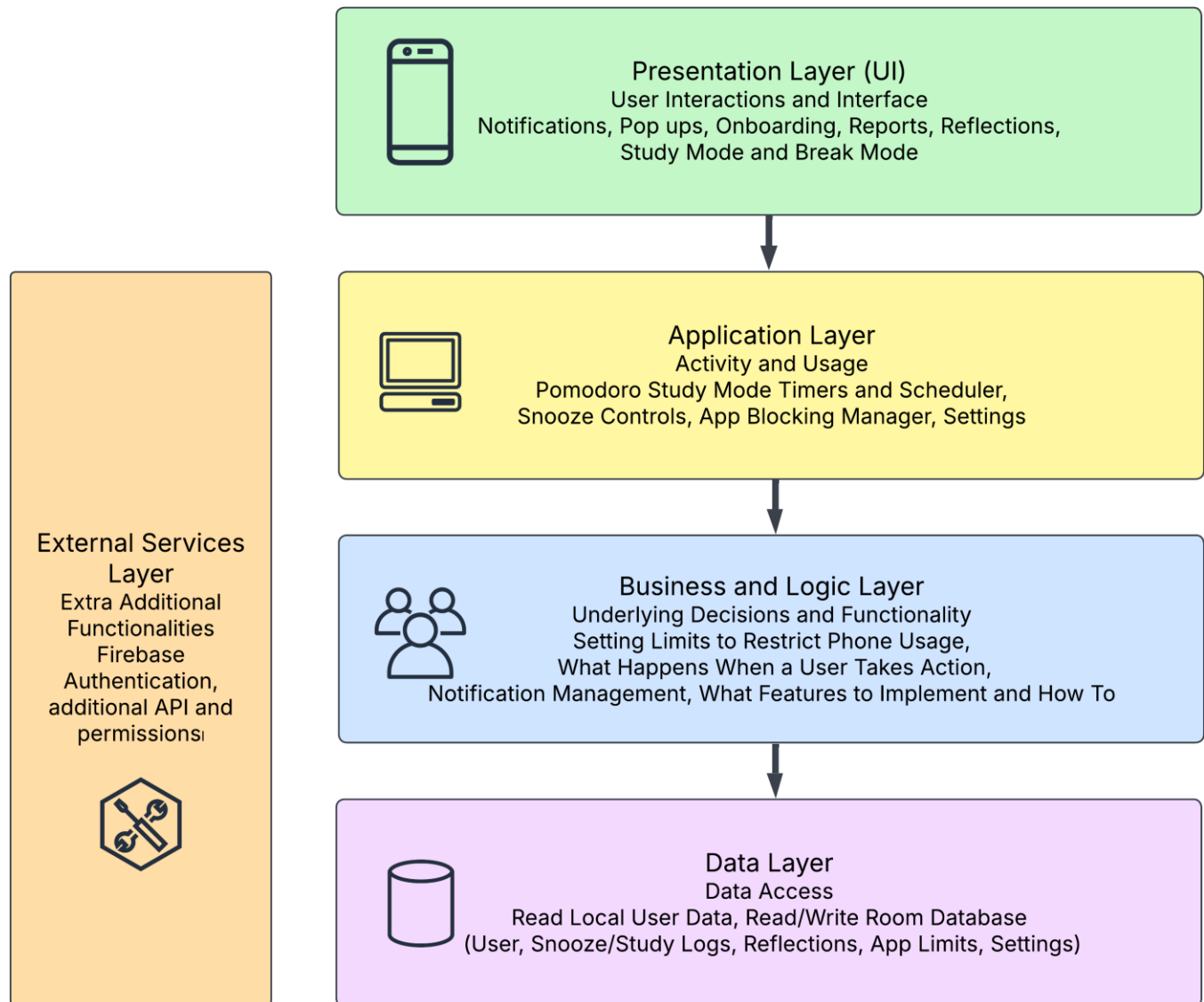


Figure A3: Illustrates the Later Gator system's layered structure, showing how the Presentation, Application, Business, Data, and Cross-Cutting layers interact to support modular development.

4. Component Decomposition Diagram

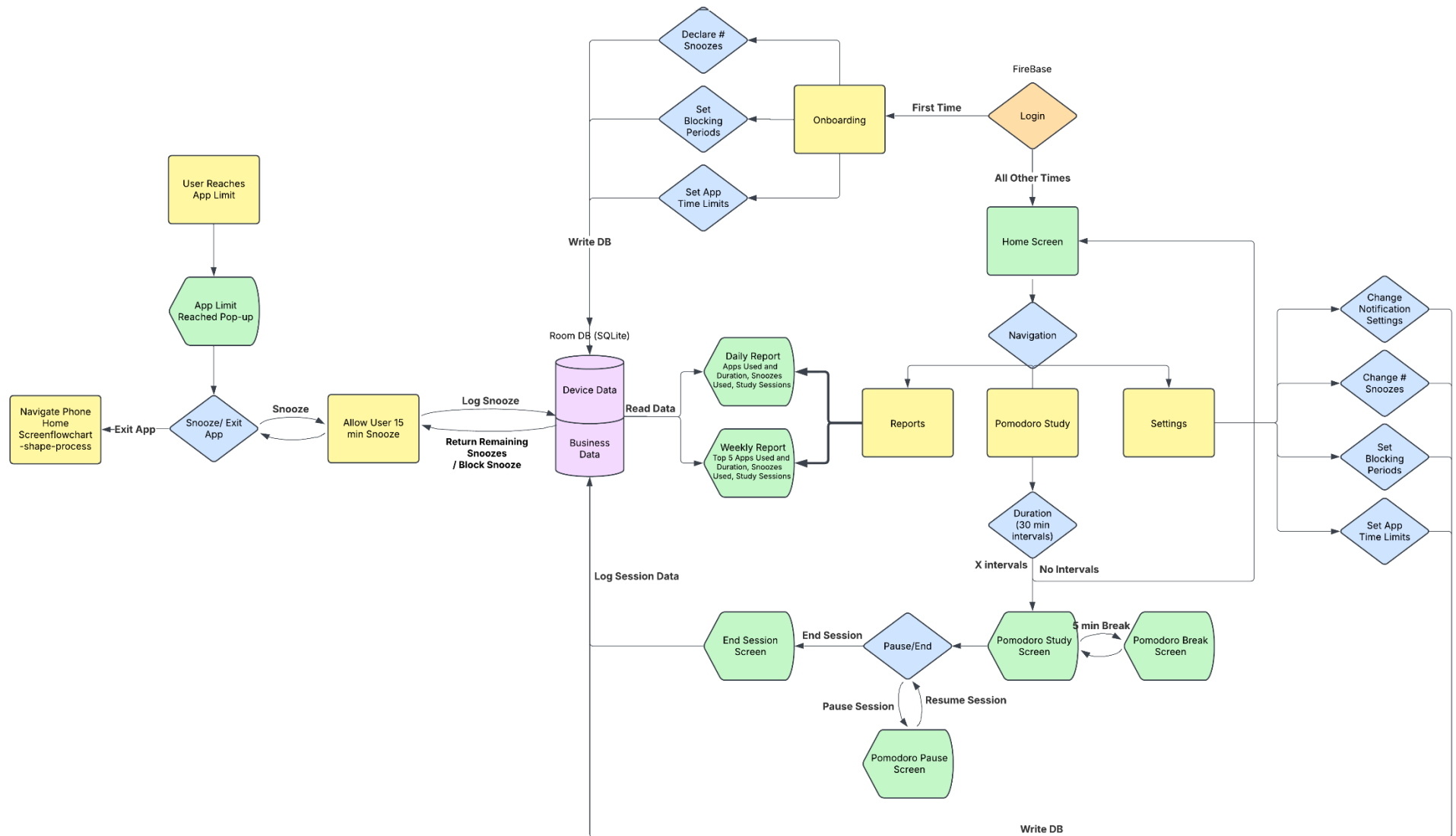


Figure A4: Finalized component breakdown displaying the relationships among system elements, their placement within architectural layers, and data flow between components.

B. System Context Model

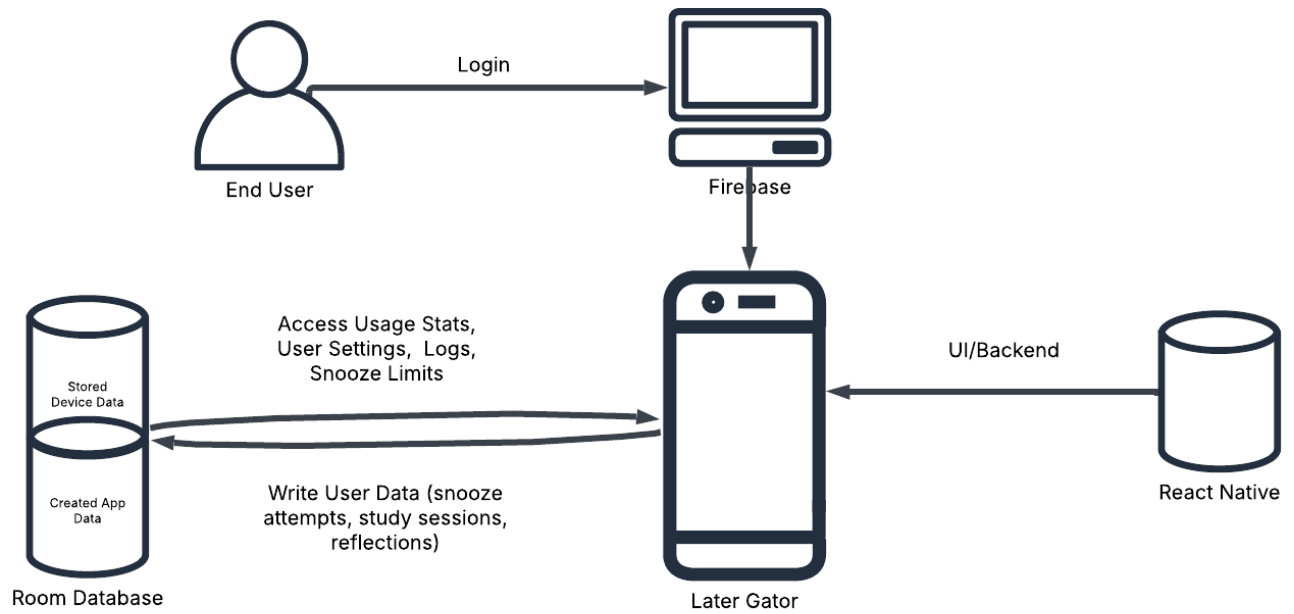


Figure B: Depicts the Later Gator system boundary and its interactions with external entities, including the user, Firebase, Room Database, and React Native framework.